

Assessed Coursework

Course Name	Internet Technology (M)		
Coursework Number	5 (of 6) – Web Application Implementation Report		
Deadline	Time: 4.30pm	Date: 19 March 2026	
% Contribution to final course mark	30	This should take at most these many hours:	15 (individual)
Solo or Group	Solo	Group	
Submission Instructions	Via Moodle – see Page 4		
Who Will Mark This?	Lecturer	Tutor	Other
Feedback Type?	Written	Oral	Both
Individual or Generic?	Generic	Individual	Both
Other Feedback Notes			
Please Note: This Coursework cannot be Re-Done			

Code of Assessment Rules for Coursework Submission

Deadlines for the submission of coursework which is to be formally assessed will be published in course documentation, and work which is submitted later than the deadline will be subject to penalty as set out below. The primary grade and secondary band awarded for coursework which is submitted after the published deadline will be calculated as follows:

(i) in respect of work submitted not more than five working days after the deadline

a. the work will be assessed in the usual way;

b. the primary grade and secondary band so determined will then be reduced by two secondary bands for each working day (or part of a working day) the work was submitted late.

(ii) work submitted more than five working days after the deadline will be awarded Grade H.

Penalties for late submission of coursework will not be imposed if good cause is established for the late submission. You should submit documents supporting good cause via MyCampus.

Penalty for non-adherence to Submission Instructions is 2 bands

Marking Criteria

See Page 9

评估作业

课程名称	互联网技术 (M)		
作业编号	5 (共6) – 网络应用实施报告		
截止日期	时间: 下午 4:30	日期: 2026年3月19日	
% 对最终成绩的贡献 课程分数	30	这最多需要这些	15 (个人)
单人或团队	Solo	团队	
提交说明	通过 Moodle – 见第 4 页		
谁会打勾?	讲师	助教	其他
反馈类型?	书面	Oral	Both
个人或通用?	通用	个人	Both
其他反馈备注			
请注意: 此课程作业不能重做			

课程作业提交的评估规则代码

正式评估的课程作业提交截止日期将公布在课程资料中, 逾期提交的作业将按照下文规定受到处罚。在公布截止日期后提交的课程作业, 其主要成绩和次要等级将按以下方式计算:

(i) 在截止日期后不超过五个工作日提交的工作

a. 该工作将按常规方式评估;

b. 据此确定的 primary grade 和 secondary band 将会因工作提交逾期每个工作日 (或部分工作日) 减少两个 secondary bands.

(ii) 在截止日期后超过五个工作日提交的工作将被评定为 Grade H。

如果因正当理由导致提交逾期, 则不会对课程作业的逾期提交处以处罚。你应该提交证明正当理由的文件, 并通过 MyCampus 提交。

未遵守提交说明的处罚为2分

评分标准

见第9页

Web Application Implementation Report (30%)

1. Introduction

The group project is based on the development of a web application of your choosing, as you described it in the Design Specification. You can make changes to the web application you develop but you will need to provide a justification for this if you do.

Your implementation should demonstrate the skills developed during the course. The back end of your application must be built using **Python and Django**, and your project must include appropriate use of **HTML and CSS**.

For the front end, you may choose your approach:

- You may use **Django templates** (server-rendered pages), *or*
- You may use a modern front-end library/framework such as **React** (or similar) and treat Django as an API back end.

You **should** also use **client-side technologies** to enhance interactivity—this can include **JavaScript**, and may include **jQuery** or **AJAX**, but **jQuery is not required** if you use React or other alternatives.

Core Functionality

Your web application should implement the main functionality described in your Design Specification (or your justified revised version).

At a minimum, your application must:

- Include user authentication (e.g., login/logout and access control)
- Interact with a database model (create/read/update/delete or other meaningful interaction)
- Allow users to input data that is used by the application (e.g., stored, processed, searched, displayed, or used to generate output)

Front-end Interactivity Requirement

Your application must not be based solely on Python/Django with static pages. There should be clear evidence of front-end/client-side functionality beyond basic server-rendered content.

This can be achieved through at least one of the following:

- JavaScript-driven interactivity (dynamic UI updates, form validation, etc.)
- Asynchronous requests (e.g. AJAX)
- A client-side framework/library used to build an interactive user interface (e.g., React or similar)

Web应用程序实现报告 (30%)

1. 简介

小组项目基于你选择并在设计规范中描述的web应用开发。你可以对开发的web应用进行修改，但如果你这样做，需要提供修改的理由。

你的实现应展示课程期间所发展的技能。你的应用程序后端必须使用Python和Django构建，并且你的项目必须包含适当的HTML和CSS使用。

对于前端，你可以选择你的方法：

- 你可以使用Django模板（服务器渲染页面），或者
- 你可以使用现代前端库/框架，如React（或类似）并处理Django 用作 API 后端。

你还应该使用客户端技术来增强交互性——这可以包括 JavaScript，也可能包括 jQuery 或 AJAX，但如果你使用 React 或其他替代方案，则不需要 jQuery。

核心功能

你的 Web 应用程序应该实现你的设计规范（或你合理修订的版本）中描述的主要功能。

至少，你的应用程序必须：

- 包含用户认证（例如登录/注销和访问控制）
- 与数据库模型交互（创建/读取/更新/删除或其他有意义的操作）
交互)
- 允许用户输入由应用程序使用的数据（例如，存储、处理、搜索、显示或用于生成输出）

前端交互需求

您的应用程序不能仅基于 Python/Django 和静态页面。应提供明确的证据表明存在前端/客户端功能，而不仅仅是基本的服务器渲染内容。

这可以通过以下至少一项来实现：

- JavaScript 驱动的交互（动态 UI 更新、表单验证等）
- 异步请求（例如 AJAX）
- 一个用于构建交互式用户界面（例如 React 或类似框架/库）的客户端框架/库

Look and Feel

- Your application should have a **polished, consistent interface** (layout, typography, spacing, navigation should feel intentional rather than clunky).
- The UI must be **responsive** across common screen sizes (mobile, tablet, desktop).
- Use a **responsive CSS framework** (e.g., Bootstrap) or an equivalent responsive approach (e.g., well-structured custom CSS using Flexbox/Grid).

Accessibility

You already created an accessibility plan in the design phase. In this project, you must show how you applied it.

- Choose 2–3 key pages/features (e.g., login + one main feature).
- Implement at least 3 improvements from your accessibility plan.

Sustainability

You must evaluate and improve the sustainability-related performance of your application using an appropriate tool (e.g., [Google Lighthouse](#) or similar).

Your sustainability report should include:

- The **tool(s) used** and the pages tested (at least **2 pages**: homepage + one core feature).
- A **baseline “before” measurement** (scores and/or key metrics, with screenshots/exported report).
- **Any changes you implemented** to improve sustainability-related performance (e.g., reducing page weight, optimising images, removing unused JS/CSS, reducing requests, caching, code-splitting).
- An **“after” measurement** showing the impact, with a short reflection on what improved and why.

Code

- Code must be **readable, well-organised, and commented where appropriate** (comment intent/decisions, not obvious lines).
- **Avoid duplication**: do not repeat large blocks of logic or markup in views, templates, or components—use reusable helpers/partials/components/utilities.
- **Separation of concerns**: CSS and JavaScript should be kept separate from templates/pages (i.e., avoid inline CSS/JS except where unavoidable and justified).
- Include **helper functions/classes/modules** where appropriate to keep views/components focused.
- Include **unit tests** covering key parts of the application (at minimum: core business logic/models; ideally one view/API endpoint). Provide clear instructions for running tests.

If using Django templates (server-rendered):

- Templates should **inherit from a base layout** where appropriate (e.g., base.html).

外观与感觉

- 您的应用程序应具有精致、一致的界面（布局、排版、间距、导航应感觉精心设计而非笨拙）。
- UI 必须在常见屏幕尺寸（移动设备、平板电脑、桌面电脑）上响应式显示。
- 使用响应式 CSS 框架（例如 Bootstrap）或等效的响应式方法（例如使用 Flexbox/Grid 的结构良好的自定义 CSS）。

无障碍访问

您在设计阶段已经创建了一个无障碍计划。在这个项目中，您必须展示如何 **you app** 撒谎了。

- 选择 2-3 个关键页面/功能（例如，登录 + 一个主要功能）。
- 至少实现您的无障碍计划中的 3 项改进。

可持续性

您必须使用合适的工具（例如，Google Lighthouse 或类似工具）评估并改进您应用的可持续性相关性能。

您的可持续性报告应包括：

- 所使用的工具(s)以及测试的页面（至少2个页面：主页 + 和一个核心功能）。
- 一个基线“之前”的测量（分数和/或关键指标，附带截图/导出的报告）。
- 为提高与可持续性相关的性能而实施的任何更改（例如，减少页面重量、优化图片、移除未使用的 JS/CSS、减少请求、缓存、代码拆分）。
- 一个“之后”的测量，展示影响，并附上简短反思，说明哪些方面得到改善以及原因。

Code

- 代码必须可读、结构良好，并在适当位置添加注释（注释意图/决策，而非明显行）。
- 避免重复：不要在视图、模板或组件中重复大量逻辑或标记——使用可重用的辅助函数/部分组件/工具。
- 关注点分离：CSS 和 JavaScript 应与模板/页面（即，避免内联 CSS/JS，除非不可避免且合理）。
- 包含辅助函数 在适当的位置包含类/模块 to keep 视图/组件应专注。
- 包含涵盖应用程序关键部分的单元测试（至少：核心业务逻辑/模型；理想情况下一个视图/API 端点）。提供清晰的测试运行说明。

如果使用 **Django** 模板（服务器渲染）：

- 模板应当根据需要继承基础布局（例如 base.html）。

- URLs must be **named and reverse-resolved** (e.g., `{% url 'route_name' %}`), not hard-coded paths.

If using a front-end framework (e.g., React):

- Follow equivalent best practices for routing, component reuse, and configuration (e.g., avoid hard-coded absolute URLs; keep API endpoints configurable).

2. What should be included in the submission

You must submit:

1. A **Written report** (PDF)
2. Your **Application code** (ZIP of the full project)
3. **Public code repository** (e.g., GitHub/GitLab)
4. **Deployed application** accessible via a public URL

You will add the URLs for **(3) the repository** and **(4) the deployed application** in your **written report**.

Your report should include the following:

1. Introduction + Links to code repository and deployed application (max ½ page)
 - Give an overview of the application and any adjustments to the design specification.
 - Please include the **URLs** to your code repository and your publicly deployed web application here.
2. Updated Design Specification (max 6 pages)
 - Provide your design specification, highlighting any changes from the original design, and provide a small justification.
 - Please include i) user stories, ii) the system architecture diagram, iii) the ER diagram, iv) the sitemap, v) wireframes.
 - *Note: You should include your design specification in your report, even if there are no changes from your original submission.*
3. Implementation Highlights (max 2 pages)
 - Describe your implementation, listing all the main components, key features, and demonstrating how the functionality, interactivity, look and feel, and code requirements are achieved.
4. Testing (max 1 page)
 - Describe the tests you have implemented for your system and demonstrate the system testing results.
5. Accessibility report (max 1 page)
 - Describe at least 3 improvements you implemented from your accessibility plan, and provide evidence of the improvements (e.g. notes, screenshots).
6. Sustainability report (max 1 page)
 - List the tool(s) used and the pages tested for sustainability. Provide a “before” baseline.
 - List the changes made to improve sustainability-related performance, provide an “after” measurement and briefly explain what improved and why.
7. Appendix: Team contributions and AI use statement (max ½ page)

- URLs 必须命名并反向解析（例如，`{% url 'route_name' %}`），而不是硬编码路径。

如果使用前端框架（例如 **React**）：

- 遵循路由、组件复用和配置的等效最佳实践（例如，避免硬编码绝对 URL；保持 API 端点可配置）。

2. 提交中应包含什么

你必须提交：

1. 书面报告（PDF）
2. 您的应用代码（完整项目的ZIP文件）
3. 公开代码仓库（例如GitHub/GitLab）
4. 可通过公共URL访问的已部署应用程序

你将在书面报告中添加（3）仓库和（4）已部署应用的URL。

您的报告应包含以下内容：

1. 简介 + 代码仓库和已部署应用程序的链接（最多半页）
 - 概述应用程序以及设计规范的任何调整。
 - 请在此处提供您的代码仓库和公开部署的 Web 应用的 URL。
2. 更新后的设计规范（最多 6 页）
 - 请提供您的设计规范，突出与原始设计的任何变更，并提供简要说明。
 - 请包括 i) 用户故事， ii) 系统架构图， iii) ER 图， iv) 网站地图， v) 线框图。
 - 注意：即使与原始提交相比没有变更，您也应在报告中包含您的设计规范。
3. 实现亮点（最多 2 页）
 - 描述你的实现方案，列出所有主要组件、关键特性，并展示功能、交互性、外观和感觉以及代码需求是如何实现的。
4. 测试（最多1页）
 - 描述你为系统实现的测试，并展示系统测试结果。
5. 可访问性报告（最多1页）
 - 描述你从可访问性计划中至少实施的3项改进，并提供改进的证据（例如笔记、截图）。
6. 可持续性报告（最多1页）
 - 列出所使用的工具和进行可持续性测试的页面。提供“之前”的基准数据。
 - 列出为提高可持续性相关性能所做的变更，提供“之后”的测量数据，并简要说明哪些方面得到改善以及原因。
7. 附录：团队贡献和AI使用声明（最多半页）

3. Generative AI (GenAI) use in this assessment

You may use Generative AI to support your work. However, if you use GenAI for anything beyond general background reading, you must **acknowledge how you used it** and **attribute it appropriately**. You remain responsible for the accuracy and originality of our submission.

When you must acknowledge GenAI use

If you use GenAI to generate content that influences your submission (e.g., debugging support, small code snippets, refactoring suggestions, test ideas, accessibility fixes, or sustainability/performance recommendations), you must include an **AI Use Statement** (see below). Where GenAI provides specific claims or technical guidance, you should verify it using reliable sources (e.g., official documentation) and by testing your implementation.

Examples

1. Debugging support

You ask GenAI to help interpret an error message or suggest likely causes and fixes (e.g., “Why am I getting CSRF verification failed in Django?” or “Why is this React state not updating?”). You should acknowledge GenAI use and ensure you understand, implement, and test the fix yourself.

2. Small code snippets / helper functions

You ask GenAI to generate or improve a short, self-contained piece of code (e.g., a helper function, a serializer method, a small validation function, a query pattern). You should acknowledge GenAI use and adapt it to your project appropriately.

3. Refactoring and code quality suggestions

You ask GenAI for suggestions to improve readability/structure (e.g., “How can I reduce duplication in these views?” or “Suggest how to structure reusable components”). You should acknowledge GenAI use and ensure the final solution reflects your own design decisions.

4. Unit test ideas / edge cases

You ask GenAI to suggest test cases or edge cases for your existing code (e.g., “What tests should I write for this model method?”). You should acknowledge GenAI use and ensure the tests are relevant, correct, and actually run.

5. Accessibility implementation guidance

You ask GenAI how to implement improvements from your accessibility plan (e.g., labels, keyboard focus, accessible error messages, ARIA for a specific component). You should acknowledge GenAI use and validate the outcome using your chosen checking tool(s).

6. Sustainability/performance improvement guidance

You ask GenAI to help interpret Lighthouse output and suggest changes (e.g., “What does ‘eliminate render-blocking resources’ mean here?”). You should acknowledge GenAI use and provide before/after evidence based on the tool results.

What is not acceptable

- Submitting GenAI-generated content **without acknowledgement**.
- Including “facts” from GenAI without checking reliability where accuracy matters.
- Presenting GenAI output as if it were an authoritative external source.

3. 本次评估中的生成式AI（GenAI）应用

你可以使用生成式AI来辅助你的工作。但是，如果你使用GenAI进行超出一般背景阅读之外的事情，你必须说明你如何使用它并恰当归因。你仍然对我们的提交的准确性和原创性负责。

何时必须承认GenAI的使用

如果你使用GenAI生成影响你提交内容（例如，调试支持），对于简短的代码片段、重构建议、测试思路、无障碍修复，或可持续性/性能建议等情况，你必须包含一份AI使用声明（见下文）。当GenAI提供具体声明或技术指导时，你应该使用可靠来源（例如官方文档）进行验证，并通过测试你的实现。

示例

1. 调试支持

你要求GenAI帮助解释错误信息或建议可能的原因和修复方法（例如，“我在Django中为什么收到CSRF验证失败？”或“我的React状态为什么没有更新？”）。你应该承认GenAI的使用并确保你理解，im

2. 小型代码片段/辅助函数

你要求生成式AI生成或改进一个简短、自包含的代码片段（例如，一个辅助函数、一个序列化方法、一个小验证函数、一个查询模式）。你应该承认使用了生成式AI，并适当地将其应用到你的项目中。

3. 重构和代码质量建议

你要求生成式AI提供改进可读性/结构的建议（例如，“如何减少这些视图中的重复代码？”或“建议如何构建可重用组件”）。你应该承认使用了生成式AI，并确保最终解决方案反映了你自己的设计决策。

4. 单元测试思路/边界案例

你向生成式AI建议为现有代码（例如，“这个模型方法应该写哪些测试？”）提供测试案例或边界案例。你应该承认使用了生成式AI，并确保测试的相关性、正确性，以及实际运行。

5. 可访问性实现指导

你向生成式AI询问如何实现可访问性计划中的改进（例如，标签、键盘焦点、可访问的错误消息、特定组件的ARIA）。你应该承认使用了生成式AI，并使用你选择的检查工具验证结果。

6. 可持续性/性能改进指导

你向生成式AI请求帮助解释Lighthouse输出并建议修改（例如，“这里的‘消除阻塞渲染的资源’是什么意思？”）。你应该承认使用了生成式AI，并根据工具结果提供前后对比证据。

什么是不被接受的

- 提交未经认可是由 GenAI 生成的内容。
- 在准确性重要的情况下，包含来自 GenAI 的“事实”而不检查其可靠性。
- 将 GenAI 的输出呈现为权威的外部来源。

- Using GenAI to generate **substantial parts of the application** (e.g., complete pages/components, full features, whole modules, full test suites) and submitting them with only minor edits.
- Asking GenAI to write your **accessibility or sustainability report** in a way that replaces your own analysis and evidence (e.g., “Write my report based on these screenshots/results”).

The university position is that using GenAI in assessments without acknowledgement may constitute academic misconduct (plagiarism).

AI Use Statement (required if GenAI used beyond background reading)

In your appendix slide, you must include an “AI Use Statement” and include any of the following statements that apply:

Declaration on the use of Generative AI:

1. We declare that we have not used GenAI in the development of this submission.
2. We declare that we have used GenAI as background reading only (concept explanations and documentation support).
3. We declare that we have used GenAI for copy-editing / improving clarity of language in the report.
4. We declare that we have used GenAI for debugging support (interpreting error messages and suggesting possible fixes), and we implemented and tested the final solution ourselves.
5. We declare that we have used GenAI to generate or refine small, self-contained code snippets (e.g., helper functions/validation), and we adapted them to our project and take responsibility for the final code.
6. We declare that we have used GenAI for refactoring and code-quality suggestions (e.g., reducing duplication, improving structure), and we implemented the changes ourselves.
7. We declare that we have used GenAI to suggest unit test ideas and edge cases, and we wrote/validated the final tests ourselves.
8. We declare that we have used GenAI to support accessibility implementation (e.g., labels, focus handling, keyboard navigation) and validated changes using checking tools (e.g., Lighthouse).
9. We declare that we have used GenAI to support sustainability/performance improvements (e.g., interpreting Lighthouse findings and suggesting optimisations) and validated changes with before/after tool results.
10. We declare that we have used GenAI in another limited way (briefly explain).

If GenAI was used, briefly add:

- the tool(s) used (e.g., ChatGPT/Copilot/Gemini)
- what parts were affected (code/tests/report)
- a 1–2 sentence summary of how you ensured correctness (e.g., testing, running lint/tests, verifying against docs)

4. How to submit

- 使用生成式AI生成应用程序的大部分内容（例如，完整页面/组件、全部功能、整个模块、完整测试套件），并仅进行少量修改后提交。
- 要求生成式AI以取代您自己的分析和证据的方式撰写您的无障碍或可持续发展报告（例如，“根据这些截图/结果撰写我的报告”）。

大学立场是，未经承认使用GenAI进行评估可能构成学术不端行为（剽窃）。

AI使用声明（若生成式AI使用超出背景阅读范围，则需提供）

在你的附录幻灯片中，你必须包含一个“AI使用声明”，并列以下适用声明：

关于使用生成式AI的声明：

1. 我们声明，我们未在本次提交的开发中使用生成式AI。
2. 我们声明，我们仅将生成式AI用作背景阅读（概念解释和文档支持）。
3. 我们声明，我们已使用 GenAI 进行报告的校对/改进语言清晰度。
4. 我们声明，我们已使用 GenAI 进行调试支持（解释错误信息并建议可能的修复方案），并且我们自行实施了并测试了最终解决方案。
5. 我们声明，我们已使用 GenAI 生成或完善小型、自包含的代码片段（例如辅助函数/验证），并且我们已将其适配到我们的项目并负责最终代码。
6. 我们声明，我们已使用 GenAI 进行重构和代码质量建议（例如减少重复、改进结构），并且我们自行实施了这些变更。
7. 我们声明，我们已使用 GenAI 建议单元测试思路和边界情况，并且我们自行编写/验证了最终测试。
8. 我们声明，我们已使用 GenAI 支持无障碍性实现（例如标签、焦点处理、键盘导航），并使用检查工具（例如 Lighthouse）验证了变更。
9. 我们声明，我们已使用 GenAI 支持可持续性/性能改进（例如，解释 Lighthouse 结果并建议优化），并使用前后工具结果验证了变更。
10. 我们声明，我们已以另一种有限的方式使用 GenAI（简要说明）。

如果使用了 GenAI，请简要添加：

- 使用的工具（例如，ChatGPT/Copilot/Gemini）
- 受影响的部分（代码/测试/报告）
- 确保正确性的 1-2 句话总结（例如，测试、运行 lint/测试、与文档核实）

4. 如何提交

Your Implementation Report must be saved as a PDF document, and your web application code must be saved in a zip file. One member of the team should submit the pdf document and the zip file via the “Implementation” submission area on the Moodle page of the course. The person making the submission will be required to complete a **Declaration of Originality** on behalf of all team members when submitting via Moodle. If you have used any external sources, be sure to acknowledge them in your submission.

5. Recommendations for deployment

There are several platforms that offer free, free-tier, or free-trial hosting suitable for Django apps. Keep in mind that the specifics of these free tiers can change over time, so it's a good idea to check the latest details on each provider's website. Here are some options:

- [PythonAnywhere](#) offers a free beginner's plan, although it comes with some limitations such as restricted outbound internet access from your apps.
- [Render.com](#) offers a free hobby plan, although it comes with some limitations regarding uptime.
- [Replit.com](#) offers a free starter plan, although you may need to configure your environment appropriately.
- Cloud providers like the Google Cloud Platform (GCP) and Amazon Web Services (AWS) provide some free tiers for a limited period or limited usage.

6. Working in a team

You should think about how the tasks associated with the development of your team's web app should be divided. Try to ensure that activities are assigned so that every member of the team can be always involved. A possible breakdown of responsibilities for a team might be (this is just an example, and you should feel free to organize things differently if you prefer):

- user authentication, unit testing
- models
- views
- templates and CSS
- Javascript, AJAX

Please note that under normal circumstances all members in a team will be assigned the same mark.

All members of a team will usually receive the same mark. Where members of a team find the contribution of other members to be unsatisfactory, you are encouraged to resolve the issue internally by negotiation first. However, if this is not possible, the course coordinators should be asked to intervene as soon as possible.

The mark awarded to individual members of the team may be adjusted to reflect the individual's contribution. **We will be using Deltas as a way of adjusting the team mark in order to arrive at an individual's mark for the team-based components of the course.**

你的实施报告必须保存为PDF文档，你的Web应用程序代码必须保存为zip文件。团队中的一名成员应通过课程Moodle页面上的“实施”提交区域提交PDF文档和zip文件。通过Moodle提交时，提交人需要代表所有团队成员完成一份原创性声明。如果你使用了任何外部资源，请确保在提交中注明。

5. 部署建议

有多个平台提供免费、免费套餐或免费试用主机，适合部署 Django 应用。请注意，这些免费套餐的具体内容可能会随时间变化，因此建议在各个提供商的网站上查看最新详情。以下是一些选项：

- PythonAnywhere 提供免费的新手计划，但存在一些限制，例如从您的应用发出的网络访问受限。
- Render.com 提供免费的业余计划，但存在关于正常运行时间的限制。
- Replit.com 提供免费的入门计划，但您可能需要适当配置您的环境。
- 云服务提供商（如 Google Cloud Platform (GCP) 和 Amazon Web Services (AWS)）提供有限期或有限使用量的免费套餐。

6. 团队协作

你应该考虑如何将与你团队网页应用开发相关的任务进行分配。尽量确保活动分配得让团队每个成员都能始终参与。一个团队可能的责任划分方式（这只是一个示例，如果你更喜欢，可以自由地以不同的方式组织事物）：

- 用户认证、单元测试
- 模型
- 视图
- 模板和CSS
- JavaScript, AJAX

请注意，在正常情况下，团队中的所有成员都会被分配相同的分数。

团队的所有成员通常会获得相同的分数。如果团队成员认为其他成员的贡献不令人满意，建议首先通过协商内部解决该问题。但是，如果无法这样做，应尽快要求课程协调员介入。

团队个别成员获得的评分可能会根据其贡献进行调整。我们将使用 Delta 作为调整团队评分的方式，以确定课程中基于团队的组成部分的个别成员评分。

Each member of a team will be required to submit a form, giving their own as well as their team members' contributions. A Delta typically adjusts the team mark up or down by 0 or more bands for a given individual according to the contribution. The computation of these Deltas will be informed by the percentage scores that each member of the team will provide, which gives a numerical estimate of the proportion of the overall effort undertaken by each person (including themselves). If in doubt, we will also require evidence of contributions, e.g. emails, GitHub commits, etc. We will also look at the contributions listed in the appendix to validate any adjustments.

每个团队成员都需要提交一份表格，说明自己的贡献以及团队成员的贡献。Delta 通常会根据个人的贡献，将团队评分上调或下调 0 或更多档位。这些 Delta 的计算将基于团队成员提供的百分比分数，这些分数提供了每个人（包括自己）所付出的总体努力比例的数值估计。如有疑问，我们还将要求提供贡献的证据，例如电子邮件、GitHub 提交等。我们还将查看附录中列出的贡献，以验证任何调整。

Sample marking scheme

The following marking scheme is intended to give a broad indication as to how marks will be apportioned. The actual marking scheme used in practice may deviate slightly from this.

Category	Marks
Deployment	
Application is deployed and can be used by marker	3
Repo is public and contains the full project code (reasonable structure, commit history evident).	2
Core Functionality	
Main functionality has been implemented reflecting the design	5
User authentication is included	5
Meaningful database interaction is included	5
User input is processed/stored/searched/displayed meaningfully	5
Front-end Interactivity	
Clear evidence of client-side interactivity beyond static server-rendering	5
Look and Feel	
Polished / refined interface, not clunky	5
Uses a responsive CSS framework	5
Accessibility	
At least 3 improvements implemented from accessibility plan; evidence provided	5
Sustainability	
Uses an appropriate tool (e.g., Lighthouse) to assess at least two pages and reports before/after results, the changes made, and a brief reflection.	5
Code	
Good implementation, with good separation of concerns	5
Code is readable, clear and commented where appropriate	3
Unit tests are included	2
Report	
Good description of design, implementation and testing	7
Clear writing and structure	3
Total	70

The total mark will be converted to a band which will be the team’s mark for this component of the assessment.

评分标准示例

本评分标准旨在大致说明分数如何分配。实际使用的评分标准可能与此略有不同。

类别	分数
部署情况	
应用程序已部署，可供评分员使用	3
该仓库是公开的，包含完整的项目代码（结构合理，提交历史清晰）。	2
核心功能	
主要功能已实现，反映了设计，用户认证已包含	5
	5
包含有意义的数据库交互	5
用户输入被有意义地处理/存储/搜索/显示	5
前端交互性	
客户端交互清晰证据，超越静态服务器渲染	5
外观与感觉	
界面精致/优雅，不粗糙	5
使用响应式CSS框架	5
无障碍访问	
至少实施了可访问性计划中的3项改进；证据已提供	5
可持续性	
使用适当的工具（例如 Lighthouse）评估至少两个页面，并报告前后结果、所做的更改以及简要反思	5
Code	
实现良好，关注点分离得当	5
代码可读性强，清晰，并在适当位置进行了注释，包含单元测试	3
	2
报告	
对设计、实现和测试的详细描述	7
清晰的表达和结构	3
总计	70

总成绩将被转换为一段分数，这将作为团队在此评估部分的成绩。